DOJO CODING PATHWAYS · WORKSHOP

La Era Agéntica y la Ingeniería de Loops

Harness Engineering, Loops Autónomos y Spec-Driven Development con Claude Code

Juan Guillermo Gómez

Principal Engineer & Expert en Firebase, Google Cloud y AI

DOJO CODING

Speaker

JGG

@JGGOMEZ

Juan Guillermo Gómez

Principal Engineer & Expert en Firebase, Google Cloud y AI

- Co-leader & co-founder de **GDG Cali**
- AI & Software Advisor @ **DevHack**
- BS en Ingeniería de Sistemas & M.Sc en Ingeniería de Software
- Especialista en Arquitectura Cloud, Sistemas Multi-Agente y Harness Engineering

DOJO CODING

$ hoy

El plan de esta sesión

1	El Espejismo de la Velocidad y las Tres Deudas Invisibles	20 min
2	Harness Engineering y la Ecuación Agéntica (Model + Harness)	25 min
3	Ingeniería de Loops Autónomos y el Principio del Trinquete (Ratchet)	25 min
4	Spec-Driven Development (SDD) con Conductor y Claude Code	20 min

DOJO CODING

Diagnóstico

El Espejismo de la Velocidad

! LA ILUSIÓN DE LA RAPIDEZ

Generar código con IA es instantáneo, pero el **razonamiento y la auditoría humana** se mantienen en un ancho de banda constante.

- Escribir diffs sintácticos es trivial y barato
- El cuello de botella real es la comprensión humana
- Más líneas por minuto no equivale a más valor

× EL RIESGO DEL FALSO RIGOR

Aceptar cambios solo porque **compilan o pasan tests superficiales** no es programar con rigor: es acumular deuda invisible.

- El compilador no sabe qué quería el usuario
- Aprobación masiva por fatiga de revisión
- Desconexión progresiva del equipo con el código

DOJO CODING

Riesgos Críticos

Las Tres Deudas Invisibles del Vibe Coding

01

Comprehension Debt

Addy Osmani (2026)

La brecha entre el código acumulado y lo que el equipo entiende. Si nadie entiende el software desde primeros principios, nadie lo arreglará cuando falle en producción.

02

Intent Debt

Pérdida del "Porqué"

Si no existe especificación explícita de negocio, el agente toma decisiones probabilísticas. El sistema muta hacia soluciones ad-hoc que nadie solicitó conscientemente.

03

Review Debt

Fatiga de Auditoría

La saturación de diffs enormes genera aprobación ritual. Los ingenieros aceptan PRs sin auditar contratos, degradando los estándares de seguridad y mantenibilidad.

DOJO CODING

Límites de Escala

$10 \neq 10\times$

La Tasa de Orquestación: Coordinar supera a programar

Lanzar diez agentes concurrentes no multiplica la velocidad por diez.

Crear tareas es barato; verificar, validar contratos, resolver colisiones entre archivos y mergear ramas concurrentes es sumamente costoso. La saturación aparece en el momento en que el tiempo de arbitraje y coordinación supera al tiempo de desarrollo directo.

DOJO CODING

Fundamento de Ingeniería

Infraestructura de Control: Harness & Context Engineering

Los cimientos deterministas que transforman modelos probabilísticos en sistemas de producción gobernables.

Guides (Feedforward)

Orientan antes de generar: especificaciones, tipado estricto, contratos de Clean Architecture y estilo.

Sensors (Feedback)

Evalúan después de actuar: linters, oráculos de pruebas, hooks de pre-ejecución y validadores de CI.

DOJO CODING

✦ Arquitectura

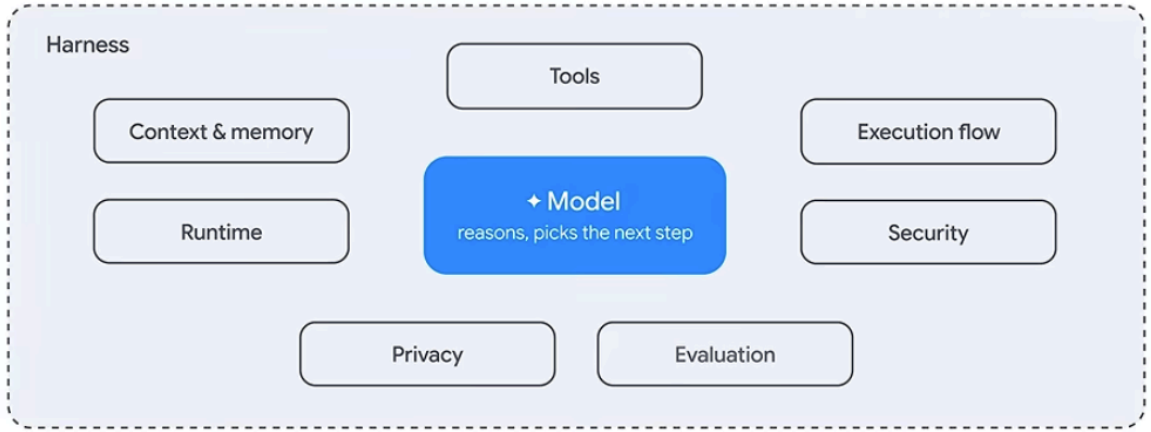
Coding Agent = AI Model + Harness (Arnés)

Guides (Feedforward)

Reglas en `CLAUDE.md`, contratos de API, especificaciones en Markdown y convenciones de código.

Sensors (Feedback)

Hooks automáticos, suite de pruebas unitarias, verificación en terminal y oráculos deterministas.



DOJO CODING

Gobernanza

El Principio del Trinquete (Ratchet)

Ante cada fallo o alucinación del agente, se programa una **regla determinista** para que ese error sea *físicamente imposible* de repetir.

Doc INSTRUCCIONES PASIVAS (CLAUDE.MD)

- Documento vivo curado por humanos con cada incidente
- Orienta el modelo antes de generar código
- **Límite:** El modelo puede ignorarlas si satura el contexto

Hook SENSORES ACTIVOS (HOOKS & PRE-COMMIT)

- Scripts en el arnés ejecutados en eventos clave
- Bloquea commits y rechaza diffs inválidos determinísticamente
- **Ventaja:** No depende de la "voluntad" del LLM

DOJO CODING

Metodología

Ingeniería de Loops Autónomos

De interactuar con prompts turno a turno a diseñar **loops recursivos en lazo cerrado** que convergen hacia una meta formal.

1

Meta Formal

Especificación exacta y criterios de éxito no negociables.

2

Agente Actúa

Ejecuta un diff pequeño y enfocado (un solo cambio).

3

Sensor Evalúa

Tests, linters y validación determinista sin sesgo.

4

Convergencia

Itera hasta cumplir oráculo o frena por presupuesto.

DOJO CODING

Filosofía

Dark Factory vs. Lit Factory

DARK FACTORY (FÁBRICA OSCURA)

- **Control:** Totalmente delegado; nadie revisa los diffs generados.
- **Sensor:** Asume falsamente que si compila, la lógica está bien.
- **Riesgo Crítico:** Deuda de comprensión absoluta y código inauditado.
- **Resultado:** Pérdida total del control del sistema ante incidentes.

LIT FACTORY (FÁBRICA ILUMINADA)

- **Control:** Prioridad del humano en la arquitectura y especificación.
- **Sensor:** Pruebas de contrato + oráculo de verificación deliberado.
- **Beneficio:** Máxima velocidad agéntica preservando el modelo mental.
- **Resultado:** Software escalable, limpio y comprendido desde el día 1.

DOJO CODING

Roles & Control

Bucle Interno vs. Bucle Externo

IA BUCLE INTERNO

El Cómo Sintáctico

- Escribir boilerplate y funciones concretas
- Resolver errores sintácticos y de compilación
- Ejecutar comandos de terminal y refactoring local
- Motor probabilístico incansable acotado por fronteras

HUM BUCLE EXTERNO

El Qué Semántico

- Definir el problema de negocio y casos de uso
- Diseñar contratos de interfaces y arquitectura limpia
- Crear la suite de verificación y oráculos de prueba
- Auditar y dar el visto bueno final para producción

DOJO CODING

Metodología SDD

Spec-Driven Development con Conductor

"Medir dos veces, programar una." Transforma a Claude Code en un ejecutor disciplinado.

1

Context Setup

`/conductor:conductor-setup`

Audita el repositorio y define la verdad única: `product.md`, `tech-stack.md` y `workflow.md`.

2

Spec & Plan

`/conductor:conductor-new-track`

Define el requerimiento en `spec.md` y un desglose atómico en `plan.md` por fases verificables antes de codificar.

3

Implement Loop

`/conductor:conductor-implement`

Claude Code ejecuta tarea por tarea, actualizando el progreso y validando contra el plan sin desviaciones.

DOJO CODING

Roadmap

Fases de Madurez en Adopción Agéntica

1

Calibración

Formar el modelo mental previo, acotar el alcance y limitar el trabajo en progreso (WIP) a tareas atómicas.

2

Scaffolding

Configurar `CLAUDE.md`, reglas de stack, sensores deterministas pre-commit y directrices de arquitectura.

3

Orquestación

Implementar arquitectura Maker/Checker, subagentes especializados y metas desacopladas en lazo cerrado.

4

Gobernanza

Optimización de ventanas de tokens, control de costos, guardrails de seguridad y observabilidad de decisiones.

DOJO CODING

Competencias Clave

Mapa de Habilidades en Ingeniería de IA (Andrew Ng)

1. Construcción y Despliegue

Gestionar la impredecibilidad del modelo frente al software tradicional. Integrar LLMs, herramientas locales y APIs agénticas.

2. Harness & Context Engineering

Diseño de prompts deterministas, reglas pasivas, gestión estricta del context window y protocolos MCP.

3. Evaluación Sistemática

Oráculos infalsificables, LLM-as-a-judge, benchmarks cerrados y validación de regresión automatizada.

4. Orquestación Multi-Agente

Coordinación de agentes especializados, aislamiento en Git Worktrees y verificación independiente de contratos.

DOJO CODING

Referencias

Referencias y Lecturas Fundamentales

01	**Comprehension Debt**	Addy Osmani (2026) · La brecha cognitiva creciente entre el código acumulado y la comprensión real del equipo.
02	**Building Effective Agents**	Anthropic Research · Por qué los arneses de herramientas y la orquestación estructurada superan a los prompts monolíticos.
03	**Spec-Driven Development**	Google / Gemini Extensions · Conductor: cómo transformar agentes de codificación en gestores de proyectos rigurosos.

DOJO CODING

¡Muchas Gracias!

Construyendo el futuro de la ingeniería con IA

Juan Guillermo Gómez Torres

Principal Engineer & Expert en Firebase, Google Cloud y AI

📍 Co-leader @ **GDG Cali**

💡 Advisor @ **DevHack**

✉️ juan.gomez01@gmail.com

🔗 linkedin.com/in/jggomez

github.com/jggomez